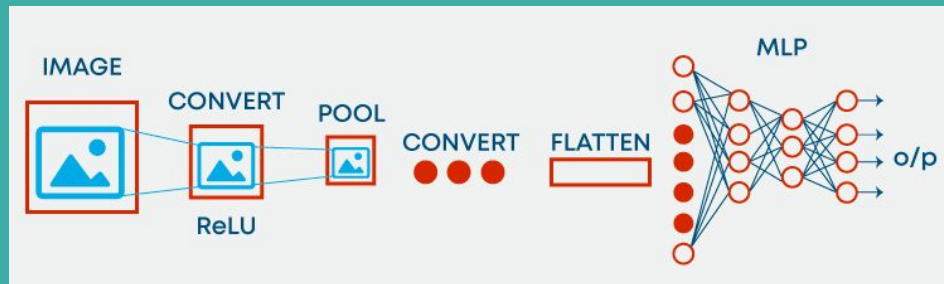


Deep Learning

CSE641/ECE555

Lecture 4 | Take your own notes during lectures

Vinayak Abrol <abrol@iiitd.ac.in>

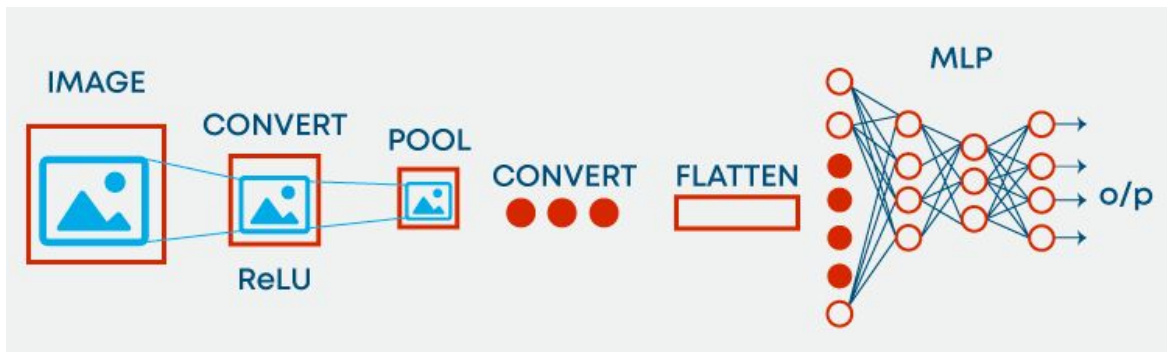


INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**

Recap: Architectures

Building Blocks

- Type of Layer **MLP/CNN/LSTM**
- Type of Model **DeepFF/Block/NiN/Branch/Residual**
- Activation Function (non-linearity) **ReLU/Tanh/Sigmoid**
- Normalization Modules **BN/LN/GN**
- Regularization Modules **L1/L2/Dropout**



Activation Functions

Parametric

$$\phi(x) = \lambda[x]_+ + \beta[-x]_+ = \begin{cases} \lambda x & \text{if } x \geq 0 \\ \beta x & \text{otherwise} \end{cases}; \quad \lambda, \beta \in \mathbb{R}$$

Rectified Linear Unit (**PReLU**)

$$\text{CReLU} = [\max(0, x), \max(0, -x)]$$

- ReLU $\max(0, x)$ corresponds to $\lambda = 1$ and $\beta = 0$, while
- Leaky-ReLU corresponds to $\lambda = 1$ and a small fixed value for β

The parameter β can be optimized during training (PReLU), or randomized for every sample (RReLU)

Activation Functions

Parametric

$$\phi(x) = \lambda[x]_+ + \beta[-x]_+ = \begin{cases} \lambda x & \text{if } x \geq 0 \\ \beta x & \text{otherwise} \end{cases}; \quad \lambda, \beta \in \mathbb{R}$$

Rectified Linear Unit (**PReLU**)

$$\text{CReLU} = [\max(0, x), \max(0, -x)]$$

- ReLU $\max(0, x)$ corresponds to $\lambda = 1$ and $\beta = 0$, while
- Leaky-ReLU corresponds to $\lambda = 1$ and a small fixed value for β

The parameter β can be optimized during training (PReLU), or randomized for every sample (RRReLU)

Parametric

Exponential Linear Unit (**PELU**)

$$\phi(x) = \lambda \begin{cases} x & \text{if } x \geq 0 \\ \alpha(e^x - 1) & \text{otherwise} \end{cases}; \quad \lambda, \alpha \in \mathbb{R}$$

- ELU corresponds to $\lambda = 1$ and $\alpha = 1$
- SeLU corresponds to $\lambda = 1.05$ and $\alpha = 1.67$

$$\text{GeLU}(x) = x\sigma(1.702x) \quad \text{Swish}(x) = x\sigma(x)$$

Saturating

Sigmoidal Class

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\text{Tanh}(x) = 2\sigma(2x) - 1$$

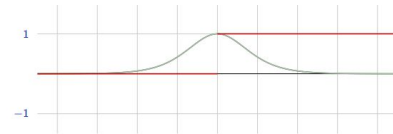
$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

Piecewise Linear

$$\text{HTanh}(x) = [x + 1]_+ - [x - 1]_+ - 1$$

Activation Functions

$$\frac{\partial \mathcal{L}}{\partial y^l} = w^{l+1} [\sigma'(z^l) \odot \frac{\partial \mathcal{L}}{\partial y^{l+1}}]$$



ReLU brought great improvement compared to Tanh for training deeper networks

(Glorot et al., 2011; Krizhevsky et al., 2012)

This can be explained by the derivative of ReLU itself not vanishing, and by the resulting coding being sparse.

The derivative of tanh has an exponential tail on both sides and collapses to 0 very quickly, while ReLU keeps the gradient of positive activations unchanged, which often correspond to half of them.

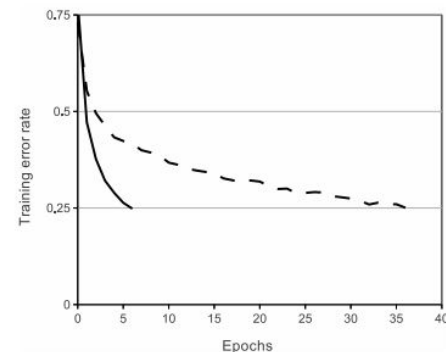
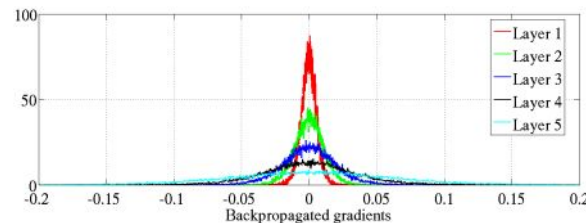


Figure 1: A four-layer convolutional neural network with ReLUs (**solid line**) reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with tanh neurons (**dashed line**). The learning rates for each net-

Remember! Deeper the model, farthest the gradient has to flow back

Activation Functions

ReLU brings
for training
(Glorot et al)

This cannot
vanish
sparse.

The derivatives
sides are
keeps the
which of

Recall **Function Approximation (UAT)** Our Target functional

Mathematically for a good approximation

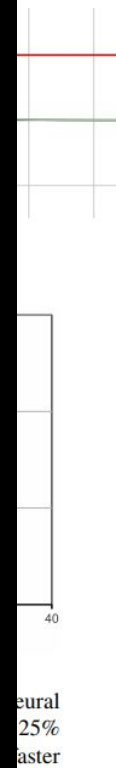
- Smooth activation for smooth functions
- ReLU like activation for Piecewise Linear functions
- ELU like activation for complicated functions

Infact Tanh like functions are theoretically (and empirically to some extent) better for very deep networks (**If used properly much better than ReLU**). But there are other ways around ReLU by changing the architecture itself!

More in TDL course

Remember! Deeper the model, farthest the gradient has to flow back

than an equivalent network with tanh neurons
(dashed line). The learning rates for each net-



Reading

- D. Clevert, T. Unterthiner, and S. Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). CoRR, abs/1511.07289, 2015.
- X. Glorot, A. Bordes, and Y. Bengio. Deep sparse rectifier neural networks. In International Conference on Artificial Intelligence and Statistics (AISTATS), 2011.
- I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. In International Conference on Machine Learning (ICML), 2013.
- K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. CoRR, abs/1502.01852, 2015.
- A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In Neural Information Processing Systems (NIPS), 2012.
- A. L. Maas, A. Y. Hannun, and A. Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In in ICML Workshop on Deep Learning for Audio, Speech and Language Processing, 2013.
- W. Shang, K. Sohn, D. Almeida, and H. Lee. Understanding and improving convolutional neural networks via concatenated rectified linear units. CoRR, abs/1603.05201, 2016.
- B. Xu, N. Wang, T. Chen, and M. Li. Empirical evaluation of rectified activations in convolutional network. CoRR, abs/1505.00853, 2015.
- Hendrycks, Dan, and Kevin Gimpel. "Gaussian error linear units (gelus)." arXiv preprint arXiv:1606.08415, 2016.
- Klambauer, Günter, et al. "Self-normalizing neural networks." Advances in neural information processing systems, 2017.

Batch-Normalization (BN)

- Ensuring proper statistics of the activations and gradients is a critical issue to train deep models.
 - It was the main motivation behind Xavier's weight initialization rule. **More on this later!**

Batch-Normalization (BN)

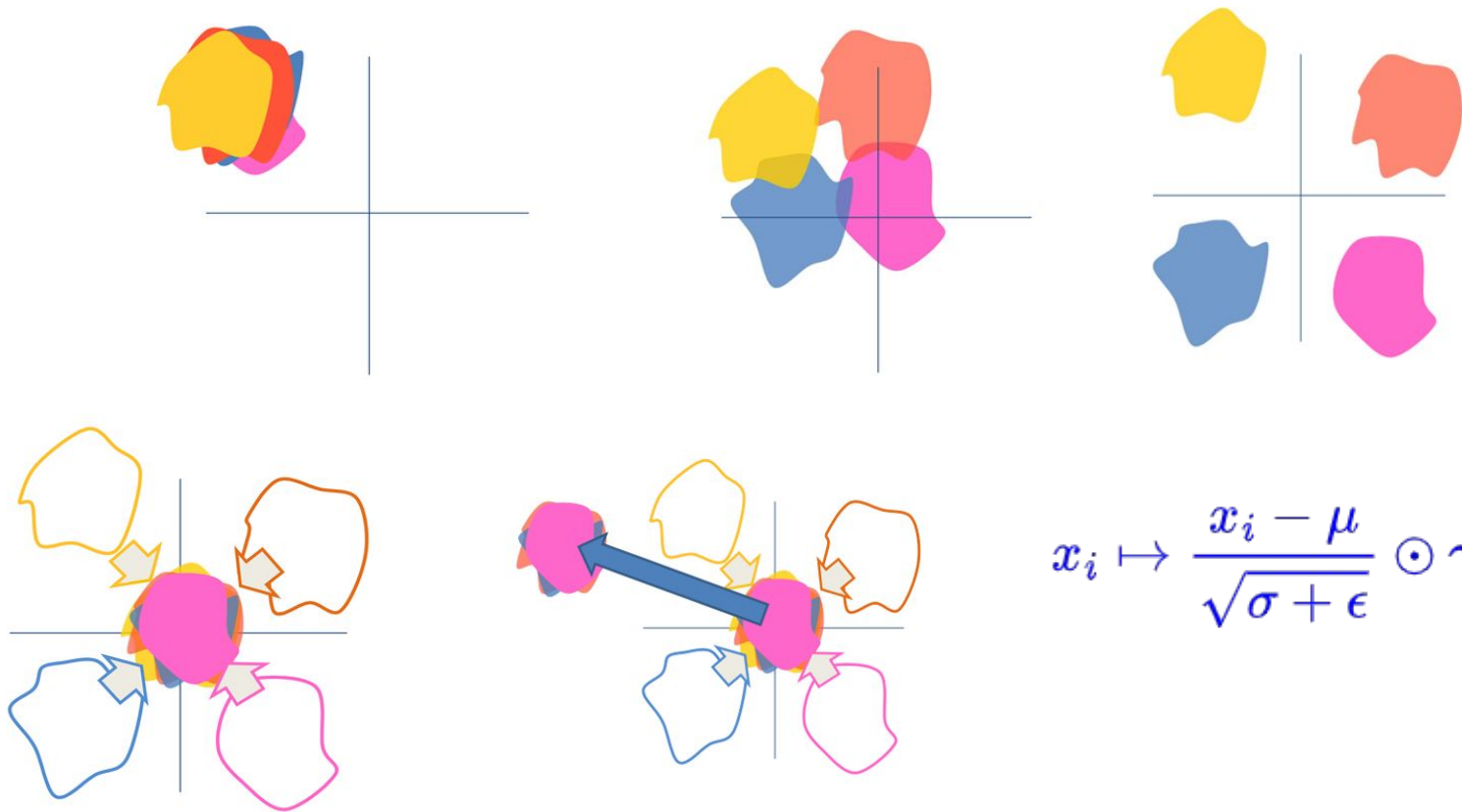
- Ensuring proper statistics of the activations and gradients is a critical issue to train deep models.
 - It was the main motivation behind Xavier's weight initialization rule. **More on this later!**
- An alternative approach to explicitly force the activation statistics during the forward pass is iterative re-normalizing.
 - **Batch normalization** Ioffe and Szegedy (2015)
“Training Deep Neural Networks is complicated by the fact that the distribution of each layer's inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization.”
 - **Training:** shifts and rescales according to the mean and variance estimated on the batch.
[If Batch Exists!]
 - **Testing:** simply shifts and rescales according to estimates from training

```
torch.nn.BatchNorm2d(num_features, eps=1e-05,  
momentum=0.1, affine=True, track_running_stats=True,  
device=None, dtype=None)
```

```
F.batch_norm(input, running_mean, running_var,  
weight=None, bias=None, training=False, momentum=0.1, eps=1e-05)
```

$$x_i \mapsto \frac{x_i - \mu}{\sqrt{\sigma + \epsilon}} \odot \gamma + \beta$$

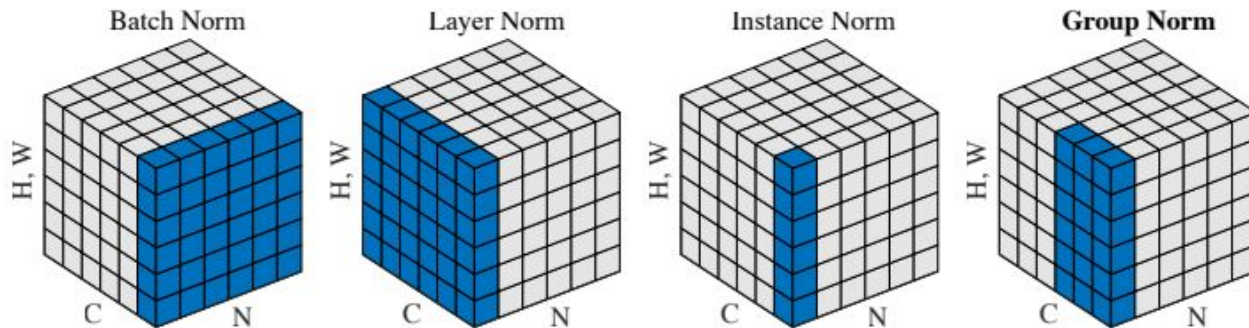
Batch-Normalization (BN)



Normalization

$$x_i \mapsto \frac{x_i - \mu}{\sqrt{\sigma + \epsilon}} \odot \gamma + \beta$$

<https://arxiv.org/pdf/1803.08494.pdf>



Normalization methods. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Normalizations invariant to weight matrix re-scaling and data re-scaling [BN, LN, IN, GN]

Normalizations invariant to weight vector re-scaling [BN, IN]

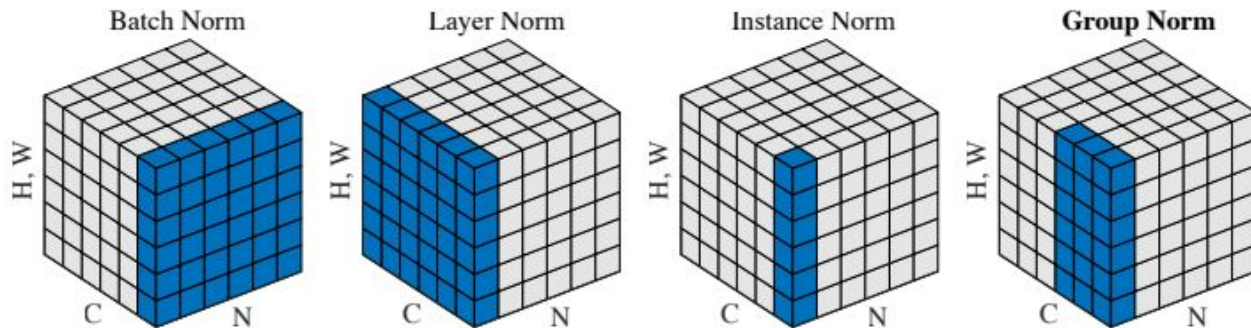
<https://arxiv.org/pdf/1607.06450.pdf>

Exercise: When not to use normalization?

Normalization

$$x_i \mapsto \frac{x_i - \mu}{\sqrt{\sigma + \epsilon}} \odot \gamma + \beta$$

<https://arxiv.org/pdf/1803.08494.pdf>



Normalization methods. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

Normalizations invariant to weight matrix re-scaling and data re-scaling [BN, LN, IN, GN]

Normalizations invariant to weight vector re-scaling [BN, IN]

<https://arxiv.org/pdf/1607.06450.pdf>

Conv $\rightarrow$ BN $\rightarrow$ ReLU

Conv $\rightarrow$ ReLU $\rightarrow$ BN

?

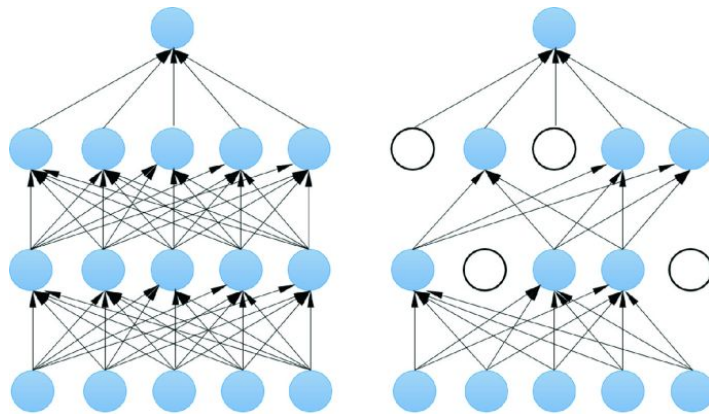
Dropout

Dropout is a **regularization** technique.

We remove units at random during the forward pass on each sample, and keep them intact during test.

"In a standard neural network, the derivative received by each parameter tells it how it should change so the final loss function is reduced, given what all other units are doing. Therefore, units may change in a way that they fix up the mistakes of the other units. This may lead to complex co-adaptations. This in turn leads to overfitting because these co-adaptations do not generalize to unseen data. **We hypothesize that for each hidden unit, dropout prevents co-adaptation by making the presence of other hidden units unreliable.** Therefore, a hidden unit cannot rely on other specific units to correct its mistakes. It must perform well in a wide variety of different contexts provided by the other hidden units."

(Srivastava et al., 2014)



(a) A standard deep neural network.

(b) After applying dropout.

Dropout

<https://jmlr.org/papers/v15/srivastava14a.html>

How to decide on which layers to use dropout?

How to decide which neuron to drop?

Bernoulli Sampling:

Let $\mathbf{N}$ be the number of neuron in a hidden layer, then the expected number of active neuron at each Dropout is $\mathbf{p} \cdot \mathbf{N}$, as we sample the neurons uniformly with probability $\mathbf{p}$.

Concretely, if we have 1000 neurons in hidden layer, if we set $p = 0.5$, then on average only 500 neurons would be active at each given time.

Let $\mathbf{X}$ be a unit activation output, and $\mathbf{D}$ be an independent Boolean random variable of probability $1 - p$. We have

$$\mathbb{E}[XD] = \mathbb{E}[X]\mathbb{E}[D] = (1 - p)\mathbb{E}[X]$$

#u is a vector of zeros/ones

```
u = np.random.binomial(1, p, size=h.shape)
```

```
u = np.random.binomial(1, p, size=h.shape)/(1-p)
```

```
#to preserve mean of layer output we divide by 1-p
```

```
h *= u
```

```
torch.nn.Dropout(p=0.5, inplace=False)
```


Numerical Stability and Parameter Initialization

$\mathbf{h}^{(l)} = f_l(\mathbf{h}^{(l-1)})$ and thus $\mathbf{o} = \mathbf{h}^{(L)} = f_L \circ \dots \circ f_1(\mathbf{x})$.

$$\begin{aligned}\text{Var}[o_i] &= E[o_i^2] - (E[o_i])^2 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2 x_j^2] - 0 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2] E[x_j^2] \\ &= n_{\text{in}} \sigma^2 \gamma^2.\end{aligned}$$

Signals in the forward pass or backward pass should not explode or vanish.

```
M = torch.normal(0, 1, size=(4, 4))
for i in range(100):
    M = M @ torch.normal(0, 1, size=(4, 4))

M = tensor([[[-0.8755, -1.2171,  1.3316,  0.1357],
              [ 0.4399,  1.4073, -1.9131, -0.4608],
              [-2.1420,  0.3643, -0.5267,  1.0277],
              [-0.1734, -0.7549,  2.3024,  1.3085]]])
```

```
tensor([[[-2.9185e+23,  1.3915e+25, -1.1865e+25,  1.4354e+24],
          [ 4.9142e+23, -2.3430e+25,  1.9979e+25, -2.4169e+24],
          [ 2.6578e+23, -1.2672e+25,  1.0805e+25, -1.3072e+24],
          [-5.2223e+23,  2.4899e+25, -2.1231e+25,  2.5684e+24]])
```

$$n_{\text{in}} \sigma^2 = 1 \quad \text{Var}[o_i] = \gamma^2$$

$$\partial_{\mathbf{W}^{(l)}} \mathbf{o} = \partial_{\mathbf{h}^{(L-1)}} \mathbf{h}^{(L)} \dots \partial_{\mathbf{h}^{(l)}} \mathbf{h}^{(l+1)} \partial_{\mathbf{W}^{(l)}} \mathbf{h}^{(l)}$$

$$n_{\text{out}} \sigma^2 = 1$$

Numerical Stability and Parameter Initialization

$\mathbf{h}^{(l)} = f_l(\mathbf{h}^{(l-1)})$ and thus $\mathbf{o} = \mathbf{h}^{(L)} = f_L \circ \dots \circ f_1(\mathbf{x})$.

$$\begin{aligned}\text{Var}[o_i] &= E[o_i^2] - (E[o_i])^2 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2 x_j^2] - 0 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2] E[x_j^2] \\ &= n_{\text{in}} \sigma^2 \gamma^2.\end{aligned}$$

Signals in the forward pass or backward pass should not explode or vanish.

$$\frac{1}{2}(n_{\text{in}} + n_{\text{out}})\sigma^2 = 1 \implies \sigma = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}.$$

For $U(-a, a)$ we have $\sigma^2 = a^2/3$

$$\begin{aligned}\partial_{\mathbf{W}^{(l)}} \mathbf{o} &= \partial_{\mathbf{h}^{(L-1)}} \mathbf{h}^{(L)} \dots \partial_{\mathbf{h}^{(l)} \mathbf{h}^{(l+1)}} \partial_{\mathbf{W}^{(l)}} \mathbf{h}^{(l)} \\ n_{\text{out}} \sigma^2 &= 1\end{aligned}$$

$$U\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right).$$

Numerical Stability and Parameter Initialization

$\mathbf{h}^{(l)} = f_l(\mathbf{h}^{(l-1)})$ and thus $\mathbf{o} = \mathbf{h}^{(L)} = f_L \circ \dots \circ f_1(\mathbf{x})$.

$$\begin{aligned}\text{Var}[o_i] &= E[o_i^2] - (E[o_i])^2 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2 x_j^2] - 0 \\ &= \sum_{j=1}^{n_{\text{in}}} E[w_{ij}^2] E[x_j^2] \\ &= n_{\text{in}} \sigma^2 \gamma^2.\end{aligned}$$

Signals in the forward pass or backward pass should not explode or vanish.

$$\text{Lecun } \sigma = \sqrt{\frac{1}{n_{\text{in}}}}$$

$$\text{Xavier } \sigma = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}$$

$$\text{He } \sigma = \sqrt{\frac{2}{n_{\text{in}}}}$$

$$\begin{aligned}\partial_{\mathbf{W}^{(l)}} \mathbf{o} &= \partial_{\mathbf{h}^{(L-1)}} \mathbf{h}^{(L)} \dots \partial_{\mathbf{h}^{(l)} \mathbf{h}^{(l+1)}} \partial_{\mathbf{W}^{(l)}} \mathbf{h}^{(l)} \\ n_{\text{out}} \sigma^2 &= 1\end{aligned}$$

zeros ?

Regularizing Estimators

An effective regularizer is one that makes a profitable trade, reducing variance significantly while not overly increasing the bias.

Most regularization strategies follow this principle:

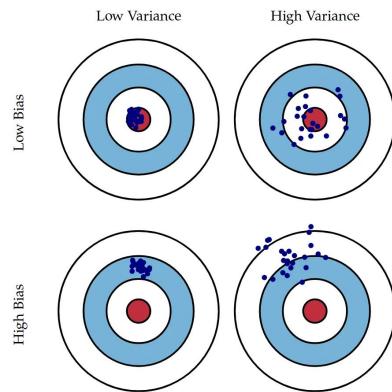
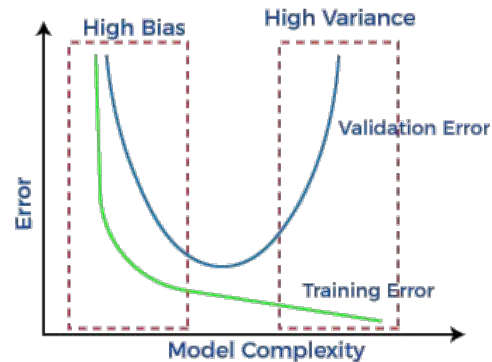


Fig. 1 Graphical illustration of bias and variance.



Regularizing Estimators

An effective regularizer is one that makes a profitable trade, reducing variance significantly while not overly increasing the bias.

Most regularization strategies follow this principle:

Forward Pass

- Norm penalties on parameters e.g., L2/L1
- Label smoothing (inject noise in targets)
- Dropout and variants (drop nodes with probability p)
- Stochastic Depth (drop layer)
- Output Normalization (Batch, Layer, Group)
-
- Early Stopping
- Parameter Sharing

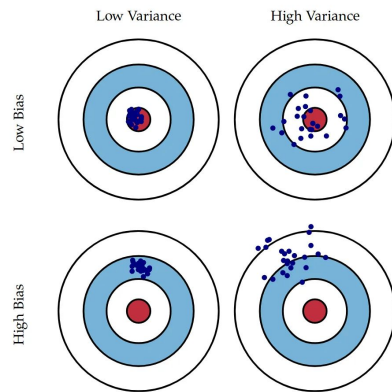
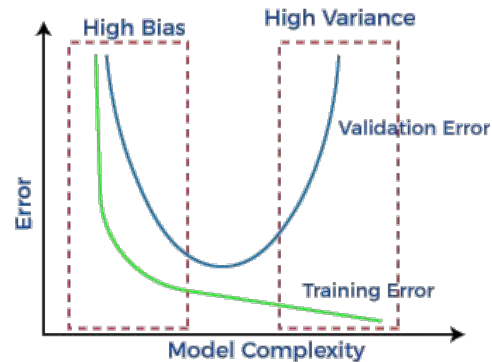


Fig. 1 Graphical illustration of bias and variance.



Regularizing Estimators

Backward Pass

- Norm penalties on gradients
 - penalizing the large output changes due to the minor input variations
 - improve model stability against adversarial or random perturbations

Most strategies? Is reducing complexity the only choice?

Exceptions [More in Meta Learning Course]

- Training the model for an auxiliary task
 - Increase complexity instead
 - Semi-supervised learning

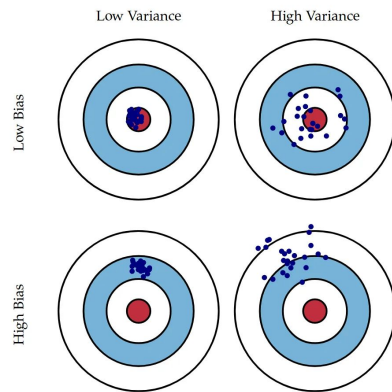
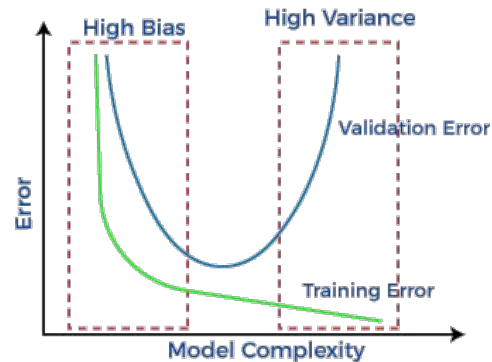


Fig. 1 Graphical illustration of bias and variance.



Regularizing Estimators [Challenges]

Existing Methods

- restrict the model's expressivity during training in static manner
 - 0.5 dropout rate always drops 50% of the hidden units
 - Dropout & variants fail for RNNs
 - weight decay always penalizes the weights by a constant factor
 - L2 norm penalty on parameters is equivalent to weight decay for SGD
 - Equivalence does not hold for adaptive gradient methods (Adam, Adadelata)
 - BatchNorm is scale invariant and cancels the effect of weight decay
 - Input gradient norms are typically averaged and less popular due to computational complexity
- are task-independent
- impact of sample complexity is still not well understood

L2/L1 Regularization

Recall MAP/MLE! Probabilistic interpretation of linear models

$$\begin{aligned}\hat{\theta}_{\text{MAP}} &= \arg \max_{\theta} P(\theta|y) \\ &= \arg \max_{\theta} \frac{P(y|\theta)P(\theta)}{P(y)} \\ &= \arg \max_{\theta} P(y|\theta)P(\theta) \\ &= \arg \max_{\theta} \log(P(y|\theta)P(\theta)) \\ &= \arg \max_{\theta} \log P(y|\theta) + \log P(\theta)\end{aligned}$$

$$\text{posterior} = \frac{\text{likelihood} \cdot \text{prior}}{\text{evidence}}$$

Ridge - L2 Penalty

Least Absolute Shrinkage and Selection Operator (LASSO) - L1 Penalty

Note

Data Samples are IID

Prior is over parameters!

L2/L1 Regularization [Gaussian Prior]

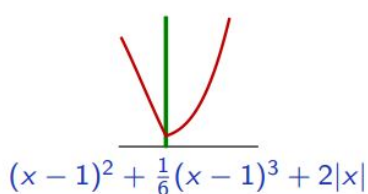
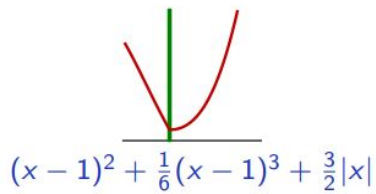
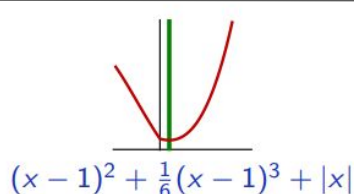
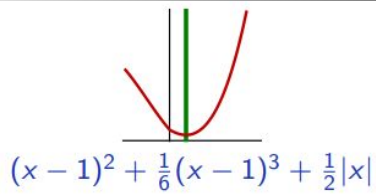
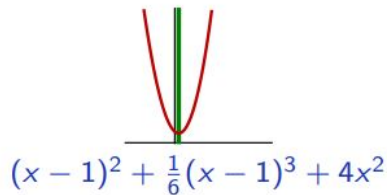
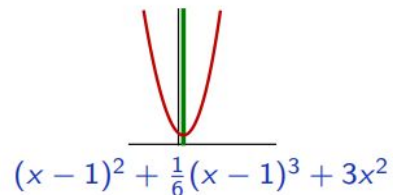
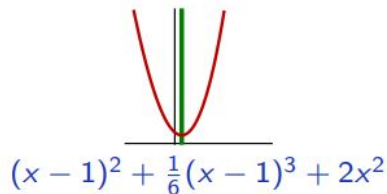
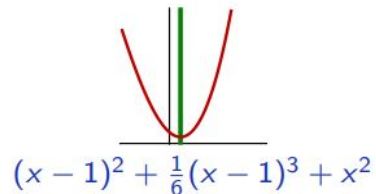
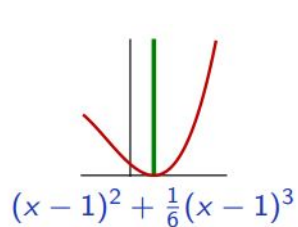
$$\begin{aligned} & \arg \max_{\mathbf{w}} \left[\log \prod_{i=1}^n \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2}{2\sigma^2}} + \log \prod_{j=0}^p \frac{1}{\tau \sqrt{2\pi}} e^{-\frac{w_j^2}{2\tau^2}} \right] \\ &= \arg \max_{\mathbf{w}} \left[- \sum_{i=1}^n \frac{(y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2}{2\sigma^2} - \sum_{j=0}^p \frac{w_j^2}{2\tau^2} \right] \\ &= \arg \min_{\mathbf{w}} \frac{1}{2\sigma^2} \left[\sum_{i=1}^n (y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2 + \frac{\sigma^2}{\tau^2} \sum_{j=0}^p w_j^2 \right] \\ &= \arg \min_{\mathbf{w}} \left[\sum_{i=1}^n (y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2 + \lambda \sum_{j=0}^p w_j^2 \right] \end{aligned}$$

L2/L1 Regularization [Laplacian Prior]

$$\begin{aligned} & \arg \max_{\mathbf{w}} \left[\log \prod_{i=1}^n \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2}{2\sigma^2}} + \log \prod_{j=0}^p \frac{1}{2b} e^{-\frac{|w_j|}{b}} \right] \\ &= \arg \max_{\mathbf{w}} \left[- \sum_{i=1}^n \frac{(y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2}{2\sigma^2} - \sum_{j=0}^p \frac{|w_j|}{b} \right] \\ &= \arg \min_{\mathbf{w}} \frac{1}{2\sigma^2} \left[\sum_{i=1}^n (y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2 + \frac{2\sigma^2}{b} \sum_{j=0}^p |w_j| \right] \\ &= \arg \min_{\mathbf{w}} \left[\sum_{i=1}^n (y_i - (w_0 + w_1 x_{i,1} + \dots + w_p x_{i,p}))^2 + \lambda \sum_{j=0}^p |w_j| \right] \end{aligned}$$

L2/L1 Regularization [Geometry]

$$w^* = \arg \min_w \mathcal{L}(w) + \lambda(\|w\|_1 \text{ or } \|w\|_2)$$



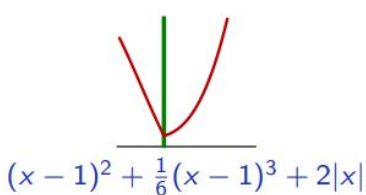
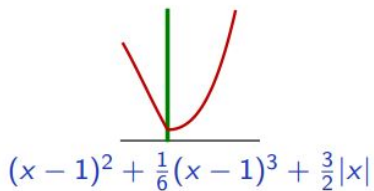
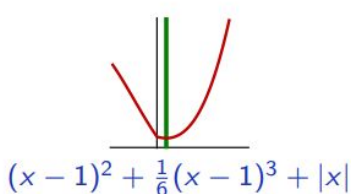
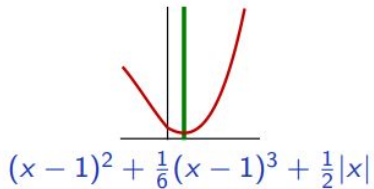
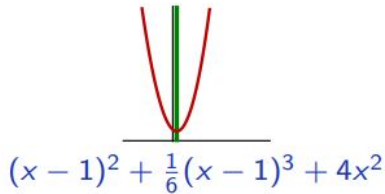
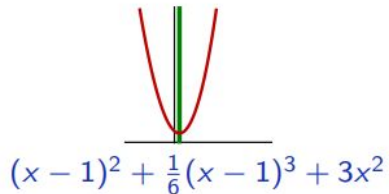
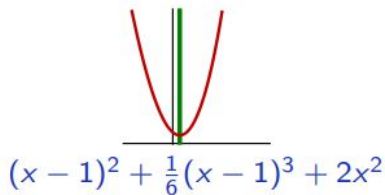
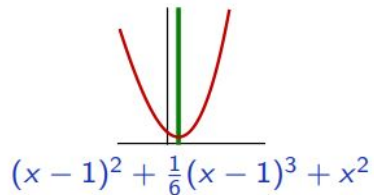
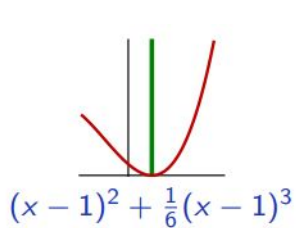
LeNet trained on MNIST with 1, 000 samples

λ	Error	
	Train	Test
0.000	0.000	0.064
0.001	0.000	0.063
0.002	0.000	0.064
0.004	0.005	0.065
0.010	0.022	0.075
0.020	0.048	0.101

λ	Error	
	Train	Test
0.00000	0.000	0.064
0.00001	0.000	0.063
0.00002	0.000	0.067
0.00005	0.004	0.068
0.00010	0.087	0.128
0.00020	0.057	0.101
0.00050	0.496	0.516

L2/L1 Regularization [Geometry]

$$w^* = \arg \min_w \mathcal{L}(w) + \lambda(\|w\|_1 \text{ or } \|w\|_2)$$

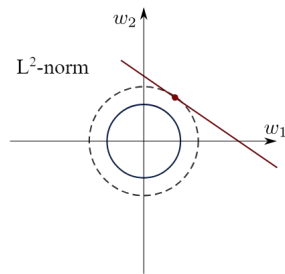
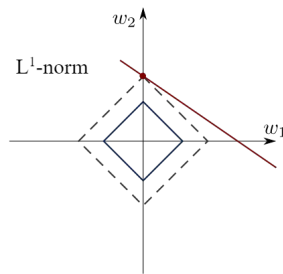


$$\mathbf{w} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right)$$

$$\mathbf{w} = (1, 0)$$

$$\|(1, 0)\|_2 = \left\| \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right) \right\|_2 = 1$$

$$\|(1, 0)\|_1 = 1 < \left\| \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right) \right\|_1 = \sqrt{2}$$

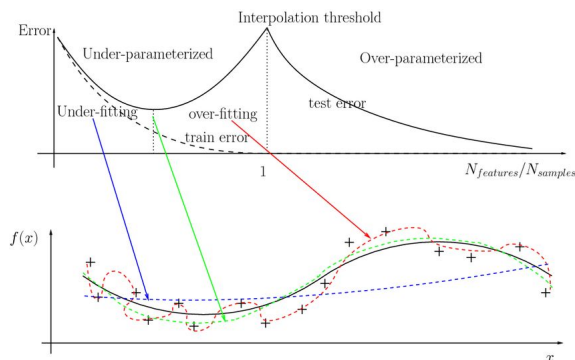


Bias-Variance Trade-off | Double Descent

The Key is in the definition of Capacity of the Model (or Hypothesis Space)

Ratio of examples vs params

The double descent phenomenon is seen in the test loss, not the training loss.



$$\sum_{r=1}^R \frac{1}{\sigma_r} (\vec{x}_{test} \cdot \vec{v}_r)(\vec{v}_r \cdot E).$$

1. Variance of training features
2. Variance & Direction of test features vary relative to the training features
3. Correlation of best model in the model class with training features

<https://arxiv.org/abs/2303.14151>

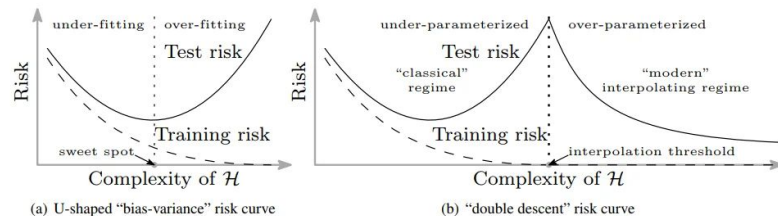
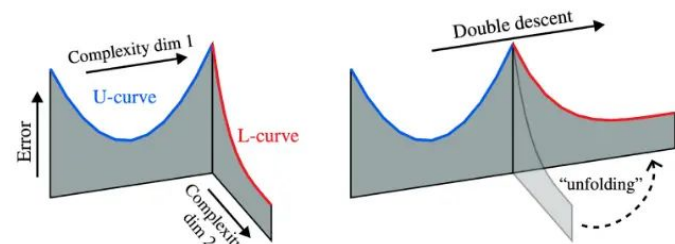


Figure 1: Curves for training risk (dashed line) and test risk (solid line). (a) The classical *U-shaped* risk curve arising from the bias-variance trade-off. (b) The *double descent* risk curve, which incorporates the U-shaped risk curve (i.e., the “classical” regime) together with the observed behavior from using high complexity function classes (i.e., the “modern” interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk.



<https://arxiv.org/abs/2310.18988>

Example: Decision Tree vs Random Forest

When number of leaves in the tree equals the number of data points, how can you increase the number of parameters?

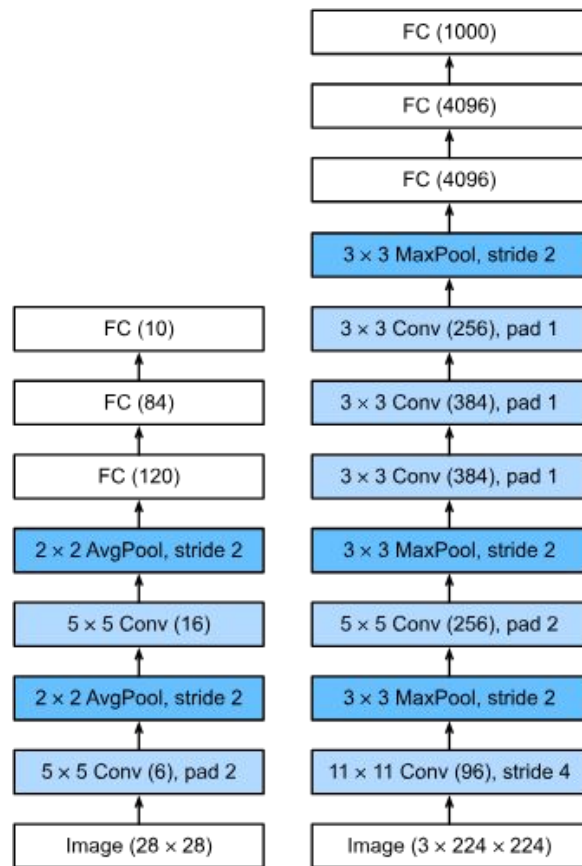
Back to CNNs: Modern CNN Architectures

Feedforward Conv-Nets

LeNet to AlexNet

- Deeper model (more layers)
- Wider model (more channels)
- ReLU instead of Sigmoid
- Larger kernel/receptive field due to input size

AlexNet was able to show significant gains on ImageNet Benchmark

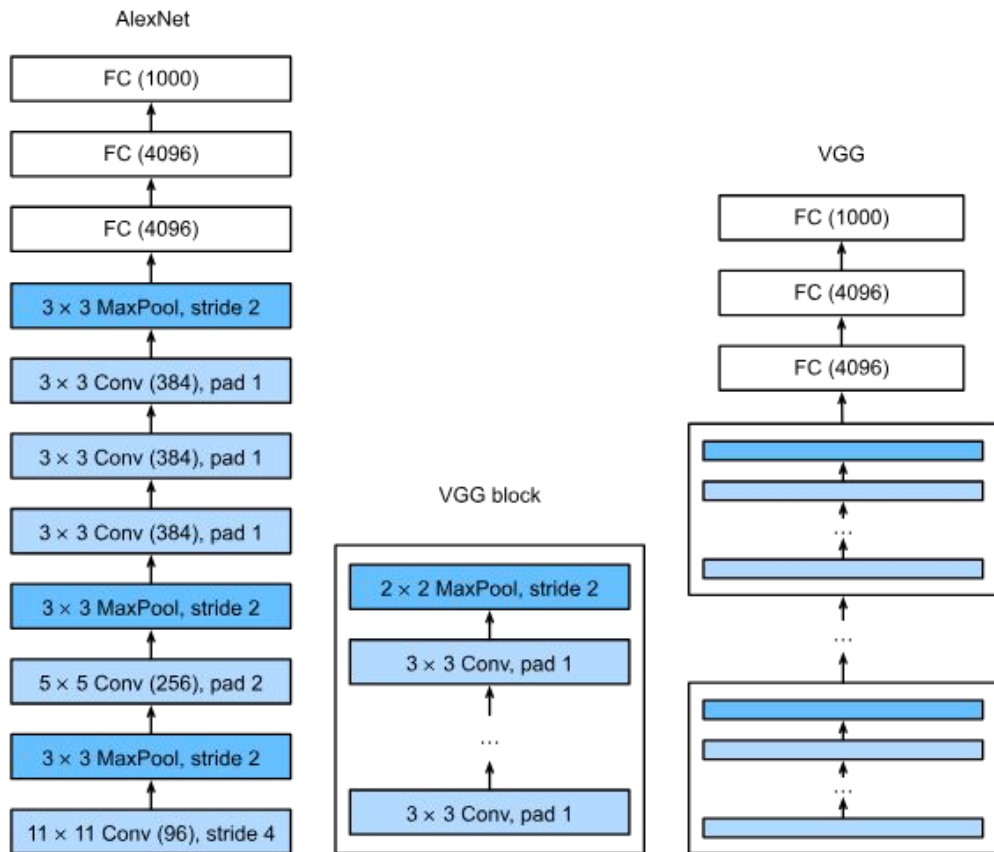


Block-Level CNNs

Deeper is better but then what to do next to improve?

- Define a template block & repeat it to desired depth
- Conv with padding (Input=Output size)
- ReLU like activation
- Pooling to reduce dimension after all convs in block
- Simulate large receptive field with small kernel over depth

3x3 followed by 3x3 conv is approx equal to 5x5 conv



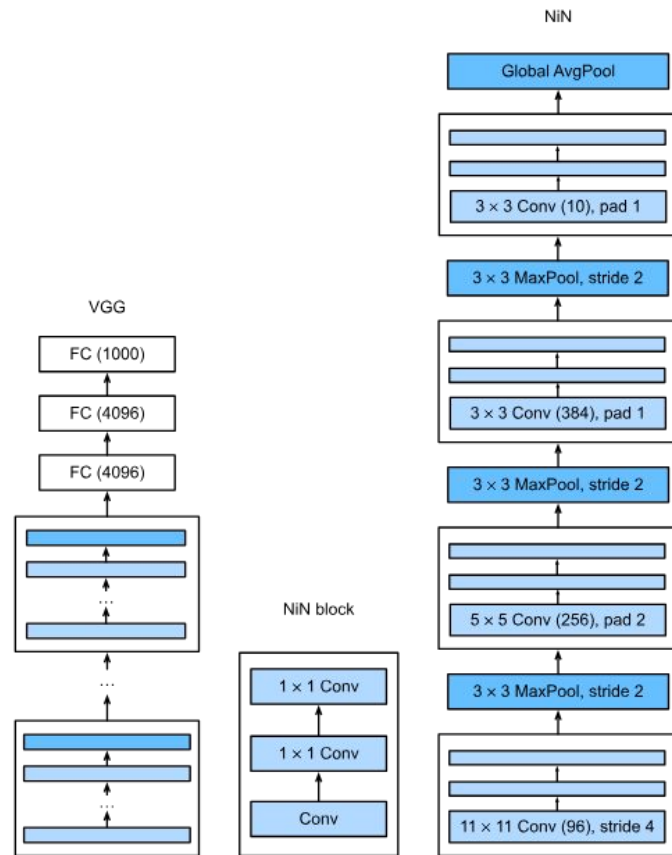
Network-in-Network (NiN) Models

Deeper is better but FC layers takes too much parameters!

The idea behind NiN is to apply a FC layer at each pixel location. It mimics AlexNet configuration.

In each block

- A regular Conv is followed by 1x1 conv
- Final NiN block uses output channels equal to classes
- Global average pooling as last layer

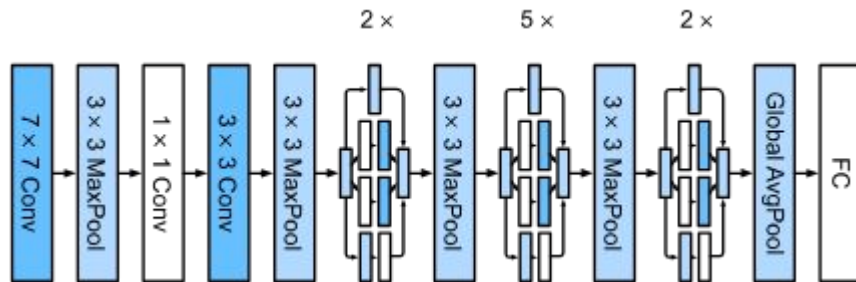
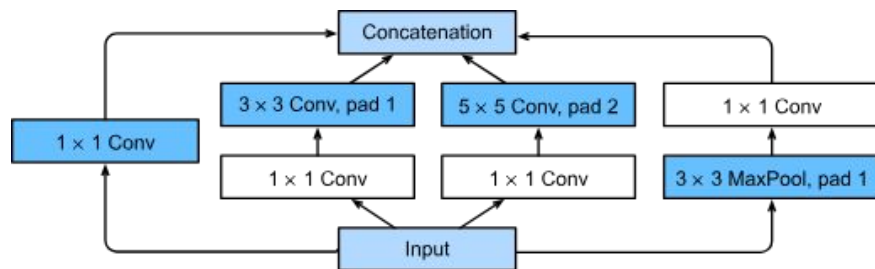


Multi-Branch Networks (GoogLeNet)

Inception Model was inspired by NiN and VGG. (And Yes Inception Movie)

The basic idea was to perform multi resolution modeling - Learn at multiple scales.

Padding is important to make tensors shapes consistent.

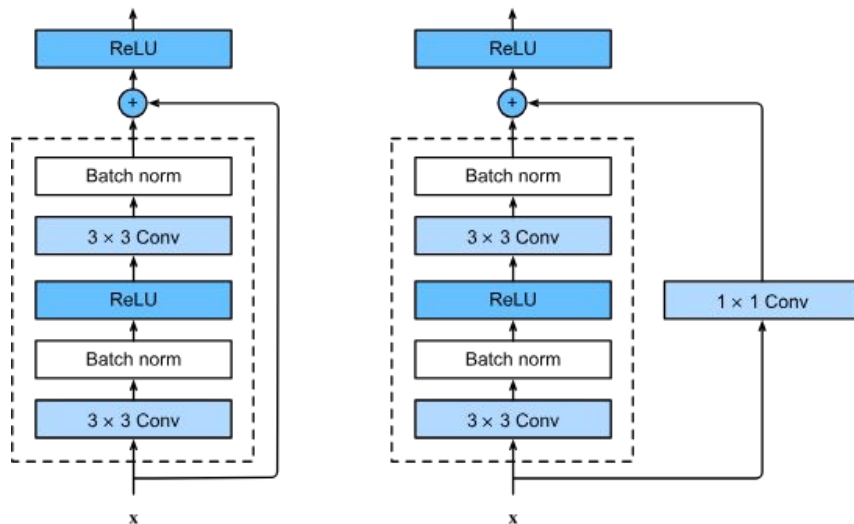
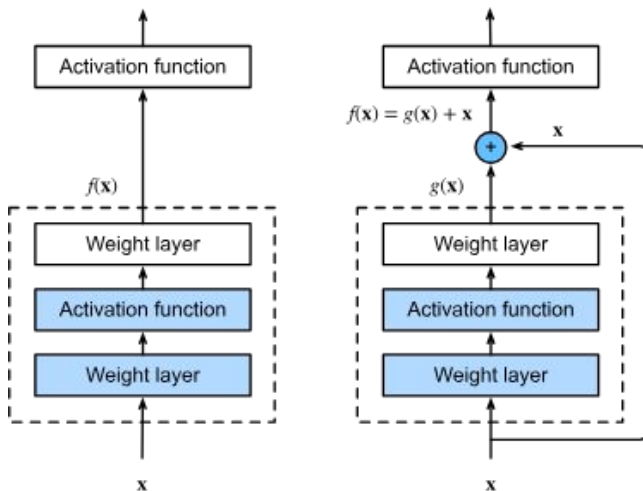


Residual Networks (ResNets)

Going Deeper became the norm but soon we hit the roadblock!

Trainability Issue

- Overparameterization
- Extensive and complex function space
- Vanishing/Exploding gradients



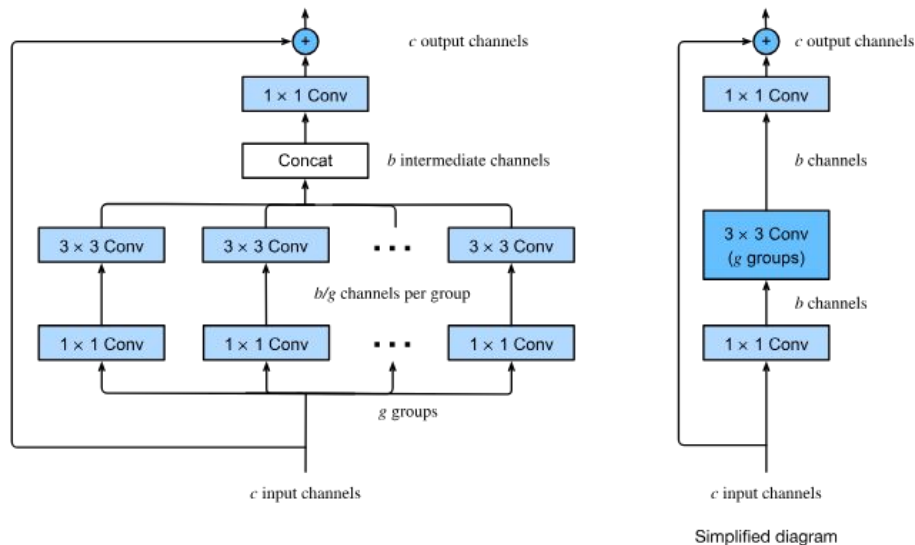
Skip Connections as a rescue path!

Also BN had become a norm with existing deep architectures like VGG.

Residual Next Networks (ResNeXts)

ResNeXt models improves over ResNet in terms of model size and performance.

Use of grouped convolution with G groups is G times faster than a dense convolution.



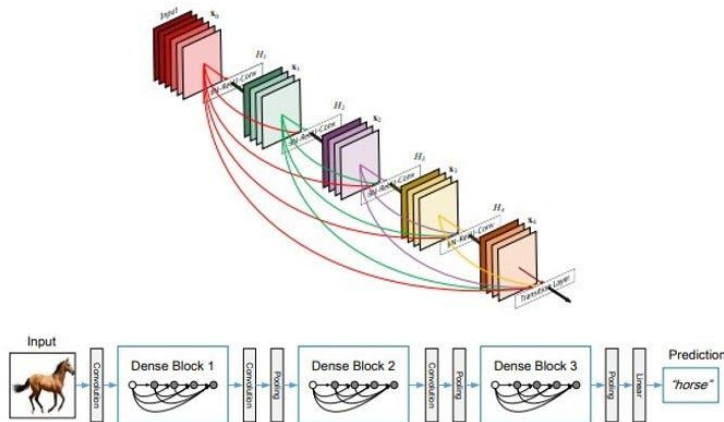
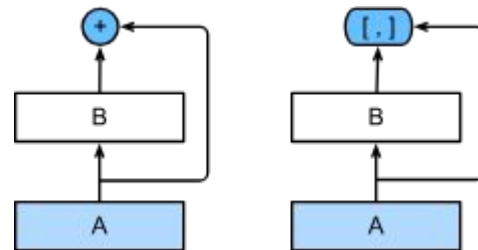
Densely Connected Networks (DenseNets)

DenseNet is characterized by both

the connectivity pattern where each layer connects to all the preceding layers

and

the concatenation operation (rather than the addition operator in ResNet) to preserve and reuse features from earlier layers.



$$\mathbf{x} \rightarrow [\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})]), f_3([\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})])), \dots]$$

Thanks

